# 5 PROGRESSO PARCIAL

Esta seção apresenta, de forma detalhada, os resultados alcançados no desenvolvimento do projeto, que seguiu uma abordagem estruturada para a implementação de Máquinas de Estados Finitos (FSMs) em Haskell[cite: 1]. A exposição segue a ordem cronológica do trabalho, evidenciando a correspondência explícita entre o código-fonte produzido e o arcabouço teórico que o fundamenta, e culmina na aplicação do framework CREM para a verificação formal das propriedades do sistema[cite: 1].

## 5.1 Correspondência entre Tipos de Dados Algébricos e os Conjuntos Q e Sigma

O primeiro resultado obtido consiste na tradução direta dos conjuntos finitos de Estados (Q) e Eventos (Σ) da quíntupla formal da FSM para Tipos Soma em Haskell, por meio do artefato `stateMachine.hs`[cite: 1]:

```
data State = Locked | Unlocked deriving (Show, Eq)
data Input = Coin | Push deriving (Show, Eq)
```

Do ponto de vista categorial, essa codificação não é uma escolha arbitrária de sintaxe, mas a instanciação direta de um resultado estrutural bem estabelecido[cite: 1]. Milewski (2020), ao apresentar a álgebra dos tipos, demonstra que o tipo `Bool`, definido em Haskell como `data Bool = True | False`, corresponde exatamente ao número 2, entendido como o coproduto de duas cópias do objeto terminal (2 = 1 + 1)[cite: 1]. Sob essa ótica, tanto `data State = Locked | Unlocked` quanto `data Input = Coin | Push` são, estruturalmente, isomorfos a `Bool`; ambos são coprodutos de dois objetos terminais na categoria Hask, a categoria cujos objetos são os tipos de Haskell e cujos morfismos são as funções entre eles (Milewski, 2020) [cite: 1]. Esse resultado confirma, em nível formal, que a finitude exigida pela definição matemática de Q e Σ é preservada com fidelidade pela representação em tipos, e não apenas sugerida por convenção de nomenclatura[cite: 1].

## 5.2 A Função de Transição delta como Morfismo em Hask

O segundo resultado é a implementação da função de transição $\delta : Q \times \Sigma \to Q$ por meio de pattern matching exaustivo[cite: 1]:

```
transition :: State -> Input -> State
transition Locked   Coin = Unlocked
transition Locked   Push = Locked
transition Unlocked Coin = Unlocked
transition Unlocked Push = Locked
```

Observa-se que a assinatura `State -> Input -> State` é a versão curried da função[cite: 1]. Milewski (2020) discute detalhadamente o processo de currying, segundo o qual uma função de múltiplos argumentos pode ser reescrita, sem perda de informação, como uma sequência de funções de um único argumento; em particular, uma função do tipo `(a, b) -> c` é equivalente, via currying, a `a -> b -> c`[cite: 1]. Isso significa que `transition` é, de fato, a representação curried de um morfismo cujo domínio é o produto categorial State × Input, a mesma noção de produto discutida por Milewski (2020) no contexto da construção universal de pares, o que estabelece uma correspondência direta e não apenas metafórica entre a assinatura de tipo do código e a definição formal $\delta : Q \times \Sigma \to Q$[cite: 1].

A verificação da Completude Funcional decorre diretamente do mecanismo de pattern matching de Haskell[cite: 1]. Hutton (2016) descreve o pattern matching como um mecanismo em que "uma sequência de padrões sintáticos é usada para escolher entre uma sequência de resultados do mesmo tipo", sendo a definição da função dada por equações que cobrem casos distintos da estrutura do argumento[cite: 1]. Mais importante para a hipótese de pesquisa é o princípio de segurança de tipos descrito por Hutton (2016) na definição do sistema de tipos de Haskell[cite: 1]:

*"Porque a inferência de tipos precede a avaliação, os programas em Haskell são type safe, no sentido de que erros de tipo nunca podem ocorrer durante a avaliação. Na prática, a inferência de tipos detecta uma classe muito ampla de erros de programa, e é uma das características mais úteis de Haskell"* (Hutton, 2016, tradução nossa)[cite: 1].

Combinado com a exigência de exaustividade do pattern matching, a ausência de qualquer uma das quatro cláusulas de `transition` é sinalizada pelo compilador GHC como um caso não tratado; esse princípio garante que todo par válido (Estado, Evento) seja necessariamente coberto pela definição de δ antes mesmo de o programa ser executado[cite: 1]. Esse resultado corrobora empiricamente a Hipótese de Completude Funcional, deslocando a verificação da cobertura lógica da FSM da fase de testes de runtime para o estágio de compilação[cite: 1].

## 5.3 Sequenciação Inicial e Validação Empírica

A composição de múltiplas transições foi inicialmente resolvida por meio da função de alta ordem `foldl`, aplicando repetidamente δ sobre uma lista de entradas[cite: 1]:

```
run :: State -> [Input] -> State
run = foldl transition
```

A execução sobre a sequência `[Coin, Push, Push, Coin, Coin, Push]`, a partir do estado `Locked`, produziu deterministicamente o estado final `Locked`[cite: 1]. Esse resultado, embora simples, é relevante porque a função `transition` utilizada aqui satisfaz a definição de função pura apresentada por Milewski (2020): "funções que sempre produzem o mesmo resultado dado o mesmo argumento de entrada, e não possuem efeitos colaterais"[cite: 1]. É justamente essa pureza — ausência de estado mutável de classe ou de variável global, na terminologia empregada por Malakhovski e Shalyto (2009) — que viabiliza a aplicação direta da verificação de tipos e da exaustividade do pattern matching como mecanismos de corretude, algo estruturalmente inacessível ao modelo imperativo tradicional[cite: 1].

## 5.4 Da Composição Ingênua à Necessidade de uma Abstração Monádica

O resultado obtido na etapa anterior é suficiente apenas enquanto a função de transição não precisa carregar nenhuma informação adicional associada a cada mudança de estado (por exemplo, uma mensagem de saída ou um registro de auditoria da transição)[cite: 1]. No momento em que essa necessidade surge, como ficará evidente na aplicação do framework CREM, ao se examinar a assinatura das ações, a função de transição deixa de ser uma função pura simples do tipo $Q \times \Sigma \to Q$ e passa a ser o que Milewski (2020) denomina uma função "embelezada" (embellished function), do tipo $Q \times \Sigma \to MQ$, em que M é um functor monádico responsável por carregar o efeito computacional associado à transição (seja ele um registro, uma possibilidade de falha, ou qualquer outro contexto)[cite: 1].

Esse foi o principal resultado teórico desta etapa: identificar, a partir do próprio artefato de código inicial, o ponto exato em que a composição via `foldl` deixaria de ser suficiente, e formalizar a solução por meio da Categoria Kleisli[cite: 1]. Segundo Milewski (2020), uma categoria de Kleisli tem como objetos os tipos da linguagem subjacente, e seus morfismos entre um tipo A e um tipo B são funções que vão de A a um tipo derivado de B por meio de uma determinada "embelezagem"; cada categoria de Kleisli define sua própria forma de compor tais morfismos, bem como os morfismos de identidade correspondentes a essa composição[cite: 1]. Formalmente, para um functor monádico M, os morfismos de Kleisli entre A e B têm o tipo $A \to MB$, com composição dada pelo operador infixo "fish"[cite: 1]:

```
(<=<) :: (a -> Writer b) -> (b -> Writer c) -> (a -> Writer c)
m1 >=> m2 = \x -> let (v, s1) = m1 x
                      (z, s2) = m2 y
                  in (z, s1 ++ s2)

return :: a -> Writer a
return x = (x, "")
```

Esse formalismo, apresentado por Milewski (2020) no contexto do Writer monad, é diretamente análogo ao problema enfrentado pelo projeto: assim como o Writer acumula uma trilha textual (log) a cada composição de funções, a FSM precisa acumular tanto o novo estado quanto uma eventual saída associada a cada transição, sem que essa responsabilidade seja delegada a uma variável mutável externa[cite: 1].

## 5.5 Reformulação Teórica da FSM sobre a State Monad

Para o tratamento específico do estado, e não apenas de um registro textual, o resultado teórico central foi mapear a FSM sobre a State Monad tal como formalizada por Hutton (2016)[cite: 1]. O autor define o transformador de estado como um tipo `newtype` que encapsula uma função pura de transformação de estado, evitando a necessidade de instâncias de classe sobre sinônimos de tipo[cite: 1]:

```
newtype ST a = S (State -> (a, State))

app :: ST a -> State -> (a, State)
app (S st) x = st x

instance Functor ST where
   fmap g st = S (\s -> let (x, s') = app st s in (g x, s'))

instance Applicative ST where
   pure x = S (\s -> (x, s))
   stf <*> stx = S (\s -> let (f, s')  = app stf s
                              (x, s'') = app stx s'
                          in (f x, s''))

instance Monad ST where
   st >>= f = S (\s -> let (x, s') = app st s in app (f x) s')
```

A partir dessa estrutura, o projeto derivou a formalização de uma transição monádica para a FSM, em que o "estado" de Hutton (2016) é instanciado pelo estado do autômato (Q) e o valor produzido a cada passo é a saída associada à transição[cite: 1]:

```
transicaoM :: Input -> ST Output
transicaoM evt = S (\q -> let q' = transition q evt
                          in (mensagem q evt, q'))
```

Essa reformulação segue exatamente o mesmo padrão estrutural do exemplo canônico de Hutton (2016) para geração de valores frescos durante o percurso de uma árvore (`fresh :: ST Int; fresh = S (\n -> (n, n+1))`), no qual um contador interno é encapsulado e roscado (threaded) de forma pura através de sucessivas aplicações, exatamente o comportamento requerido para que o estado do autômato seja propagado, transformado e extraído a cada evento da sequência, sem exposição de mutabilidade[cite: 1].

## 5.6 Sequenciação via Notação do e a Função mapM

Um segundo resultado foi a identificação, na literatura anexada, do mecanismo exato para generalizar a composição de `foldl` para o contexto monádico[cite: 1]. Hutton (2016) define, de forma genérica para qualquer Monad m, a função[cite: 1]:

```
mapM :: Monad m => (a -> m b) -> [a] -> m [b]
mapM f []     = return []
mapM f (x:xs) = do y  <- f x
                   ys <- mapM f xs
                   return (y:ys)
```

Aplicando `mapM` à função `transicaoM` sobre a lista de eventos, obtém-se a ação monádica `mapM transicaoM eventos :: ST [Output]`, cuja execução por meio de `app (mapM transicaoM eventos) Locked` produziria simultaneamente (i) a lista de saídas associadas a cada transição e (ii) o estado final do autômato, generalizando, de maneira formalmente rigorosa, o resultado obtido apenas com `foldl`[cite: 1]. A tradução dessa notação para o idioma do próprio projeto, "notação do", encontra respaldo direto em Oliveira (2017), que descreve essa construção como uma forma de reescrever expressões baseadas no operador de ligação (>>=) "de uma maneira parecida com as linguagens de programação imperativas", preservando, porém, a pureza referencial subjacente[cite: 1].

## 5.7 Fundamentação Categorial: o Monad como Monoide na Categoria dos Endofuntores

Por fim, consolidou-se a fundamentação categorial mais abstrata do projeto[cite: 1]. Milewski (2020) define formalmente um monad como um endofunctor T equipado com duas transformações naturais[cite: 1],

$$\mu : T \circ T \to T \qquad \eta : I \to T,$$

satisfazendo leis de associatividade e de unidade estruturalmente idênticas às leis de um monoide, resultado que o autor sintetiza na conhecida formulação atribuída a Saunders Mac Lane: "monad is just a monoid in the category of endofunctors" (Milewski, 2020)[cite: 1]. Esse resultado teórico é o que garante, em última instância, que a composição de transições por meio do operador fish (`>=>`) seja associativa e possua um elemento neutro (`return`), propriedade essencial para que a sequenciação de eventos da FSM produza o mesmo resultado independentemente de como a lista de eventos for particionada ou agrupada durante o processamento, generalizando com rigor formal a associatividade implicitamente exigida pelo uso de `foldl`[cite: 1].

Em síntese, o resultado desta etapa é de natureza primariamente teórica e formal: a FSM foi mapeada com precisão sobre o arcabouço da State Monad (Hutton, 2016) e da Categoria Kleisli (Milewski, 2020), estabelecendo a base necessária para a etapa seguinte[cite: 1]. Notavelmente, essa fundamentação se revelou não apenas preparatória, mas estritamente necessária: como será demonstrado na aplicação do framework CREM, o próprio framework exige, na assinatura de suas ações, um contexto `Applicative m` genérico, evidência prática direta de que a generalização monádica discutida é pré-requisito conceitual para compreender e operar a arquitetura de composição estudada[cite: 1].

## 5.8 Estudo de Caso: Topologia Promovida ao Nível de Tipos

A etapa atual do projeto aplicou o framework CREM (Composable Representable Executable Machines), de Perone (2023), ao mesmo estudo de caso da catraca, com o objetivo de investigar a Hipótese de Composabilidade e Modulação[cite: 1]. O primeiro resultado foi a codificação da topologia de transições como um valor de primeira classe, posteriormente promovido ao nível de tipos por meio do mecanismo de singletons[cite: 1]:

```
$(singletons [d|
  data TurnstileVertex = Locked | Unlocked
    deriving (Show, Eq)
|])

turnstileTopology :: Topology TurnstileVertex
turnstileTopology = Topology
  [ (Locked, [Unlocked])
  , (Unlocked, [Locked])
  ]
```

Essa construção representa uma extensão do sistema de tipos descrito por Hutton (2016), em que classes de tipos, como Eq e Ord, já restringem quais tipos podem ser usados em determinados contextos, levada a um nível mais expressivo: em vez de restringir apenas o tipo de um valor, a topologia restringe quais valores de transição são aceitáveis, tornando a regra de negócio da FSM uma propriedade do próprio sistema de tipos[cite: 1].

## 5.9 Verificação Formal de Segurança de Tipos em Tempo de Compilação

O resultado mais significativo é a verificação prática da Hipótese de Segurança de Tipos[cite: 1]. As ações da FSM foram implementadas como[cite: 1]:

```
turnstileAction :: Applicative m
  => STurnstileVertex initialVertex
  -> Input
  -> ActionResult m TurnstileTopology STurnstileVertex initialVertex Output
turnstileAction SLocked Coin = pureResult "Unlocked (insert coin)" SUnlocked
turnstileAction SLocked Push = pureResult "Locked (push denied)" SLocked
turnstileAction SUnlocked Coin = pureResult "Unlocked (already open)" SUnlocked
turnstileAction SUnlocked Push = pureResult "Locked (passed through)" SLocked
```

Note-se que a assinatura de `turnstileAction` confirma diretamente a conclusão teórica da etapa anterior: a ação é genérica sobre um contexto `Applicative m` arbitrário, e o construtor `ActionResult` exige,

implicitamente, uma prova em tempo de compilação (a restrição `AllowedTransition topology initialVertex finalVertex`) de que a transição declarada pertence à topologia especificada[cite: 1].

O princípio de type safety formulado por Hutton (2016), segundo o qual "erros de tipo nunca podem ocorrer durante a avaliação", pois a inferência de tipos precede a execução, foi aqui estendido do nível sintático para o nível semântico do domínio: uma tentativa deliberada de codificar uma transição fora da topologia declarada (por exemplo, `turnstileAction SLocked Push = pureResult "..." SUnlocked`) foi rejeitada pelo compilador GHC antes da execução do programa, por ausência da instância `AllowedTransition` correspondente[cite: 1]. O erro de tipo, nesse caso, não decorre de uma incompatibilidade estrutural entre tipos primitivos (como discutido por Hutton, 2016, no contexto geral da linguagem), mas de uma violação de regra de negócio codificada como restrição de tipo, exatamente a proposta de Perone (2023) de especificar transições permitidas e proibidas diretamente na assinatura de tipos[cite: 1].

## 5.10 Execução e Validação Cruzada

A execução do artefato (via `cabal run`, dado que o CREM é uma dependência externa) sobre a mesma sequência de entradas `[Coin, Push, Push, Coin, Coin, Push]` produziu o seguinte traço[cite: 1]:

```
Coin -> Unlocked (insert coin)
Push -> Locked (passed through)
Push -> Locked (push denied)
Coin -> Unlocked (insert coin)
Coin -> Unlocked (already open)
Push -> Locked (passed through)
```

O estado final resultante desse traço é `Locked`, idêntico ao estado final obtido de forma independente pela implementação ingênua (`run Locked inputs = Locked`)[cite: 1]. Essa coincidência não é trivial: trata-se de uma validação cruzada empírica entre duas formalizações estruturalmente distintas da mesma FSM, uma baseada em pattern matching simples com `foldl` e outra baseada em topologia promovida ao nível de tipos com efeito monádico genérico, reforçando a corretude semântica da função de transição δ modelada desde o início e corroborando a consistência do encadeamento teórico estabelecido[cite: 1].

## 5.11 Limitações Identificadas e Encaminhamentos

Em conformidade com o rigor científico exigido, registra-se uma limitação identificada na formalização atual: como o evento (Input) não está codificado no próprio vértice do tipo, a Topology declarada restringe apenas a alcançabilidade estado-a-estado ($Q \to Q$) e não o par completo $(Q \times \Sigma) \to Q$ exigido pela definição formal de δ[cite: 1]. Uma direção metodológica identificada para o refinamento dessa modelagem, a ser explorada na continuidade do trabalho, é o uso da noção de functor representável descrita por Milewski (2020), segundo a qual um functor pode ser interpretado como "um contêiner para armazenar resultados memoizados de chamadas de função", indexado por um tipo-chave[cite: 1]. Essa técnica sugere um caminho formal para codificar o alfabeto Σ como índice (chave) das transições permitidas em cada vértice da topologia, aproximando ainda mais a representação em tipos da definição matemática completa de δ[cite: 1].

Adicionalmente, a geração automática de diagramas topológicos, prevista pelo módulo Render do CREM (Perone, 2023), ainda não foi exercitada, permanecendo como resultado pendente[cite: 1].

## 5.12 Síntese Geral e Validação Preliminar das Hipóteses

A síntese dos resultados obtidos permite uma sistematização da evolução do rigor formal alcançado em cada etapa do projeto[cite: 1]. Inicialmente, na implementação ingênua, obteve-se um mecanismo de verificação de corretude baseado na exaustividade do pattern matching, com um mecanismo de composição simples via `foldl`, fundamentado teoricamente por Hutton (2016) no contexto de Tipos de Dados Algébricos[cite: 1]. Em seguida, na etapa de formalização teórica, avançou-se para a utilização da State Monad e do operador fish (`>=>`), com base na Categoria Kleisli descrita por Milewski (2020), estabelecendo as leis do monoide/monad como fundamento para a verificação de corretude[cite: 1]. Finalmente, na etapa de verificação formal com o framework CREM, alcançou-se o nível mais expressivo de segurança, com a prova `AllowedTransition` em

tempo de compilação, conforme proposto por Perone (2023), generalizando o mecanismo de composição para os construtores Sequential e Parallel[cite: 1].

Essa evolução evidencia que, enquanto nas etapas iniciais transições inválidas não eram detectáveis ou resultavam em indefinição, na etapa final tais violações são capturadas como erros de compilação[cite: 1].

Os resultados obtidos permitem uma avaliação preliminar das hipóteses de pesquisa[cite: 1]. A Hipótese de Completude Funcional foi confirmada já na implementação inicial, por meio da exaustividade do pattern matching garantida pelo compilador (Hutton, 2016)[cite: 1]. A Hipótese de Segurança de Tipos foi validada em dois níveis: um nível básico, decorrente da própria natureza type safe de Haskell (Hutton, 2016), e um nível mais profundo, específico do domínio da FSM, comprovado empiricamente por meio da rejeição, em tempo de compilação, de transições não previstas na topologia (Perone, 2023)[cite: 1]. Por fim, a Hipótese de Composabilidade e Modulação teve sua fundamentação teórica estabelecida através da Categoria Kleisli e da State Monad (Hutton, 2016; Milewski, 2020), e sua primeira instância prática obtida por meio da arquitetura genérica de `ActionResult` do CREM; a demonstração completa dessa hipótese, contudo, depende ainda da composição efetiva de múltiplas FSMs independentes por meio dos construtores Sequential, Parallel, Alternative e Feedback, etapa que permanece como objetivo central para a continuidade do trabalho[cite: 1].